

Dual-Stage Leadscrew-Driven Omni-Directional Warehouse Bot

Proposed by Team 89

Contents

1	Executive Summary	3
1.1	Problem Statement	3
1.2	Solution Summary	3
1.3	Key Metrics	4
1.4	Innovation Highlights	4
2	Problem Understanding and Motivation	5
2.1	Warehouse Challenges	5
2.2	Current Solution Gaps	5
2.3	Value Proposition	6
2.4	How We Address These Challenges	6
2.5	Competition Objectives	8
3	System Architecture	9
3.1	Overall Architecture	9
3.2	Hardware Architecture	10
3.3	Software Architecture	11
3.4	Dataflow Diagram	13
4	Hardware Design	14
4.1	Chassis Specifications	14
4.2	Drive System	15
4.3	Vertical Lift Mechanism	15
4.4	Sensor Suite and Justification	16
5	Software Architecture	18
5.1	Scanning Pipeline	18
5.2	Navigation Pipeline	21
5.3	QR Code Acquisition	24
5.4	HMI Design	24
5.5	Communication Protocols	25
6	Integration and Control	27
6.1	Hardware Software Interaction	27
6.2	Fail-Safe Mechanisms	27
7	Testing and Validation Summary	29
7.1	Methodology Overview	29
7.2	Key Performance Metrics	29
7.3	Test Results Summary	30

8	Innovation and Novel Contributions	31
8.1	Unique Design Choices	31
8.2	Algorithm Optimizations	31
8.3	Cost and Efficiency Innovations	32
9	Limitations and Future Scope	34
9.1	Known Limitations	34
9.2	Commercialization Path	35
9.3	Scalability	36
10	Innovation Highlights	38
10.1	Innovative Approaches	38
10.2	Algorithms Used	38
11	Testing and Validation	39
11.1	Structure of Testing	39
12	User Manual Details	41
13	Maintenance	43
14	References	45
14.1	ROS packages	45
14.2	Libraries Used	45

Chapter 1

Executive Summary

1.1 Problem Statement

The objective is to design and implement a comprehensive autonomous mobile robot system capable of reliably navigating indoor warehouse environments and performing vertical scanning operations on storage racks up to 2 meters in height. The system must capture high-resolution visual data from bottom to top (or vice versa), ensure sharp focus with minimal motion blur, detect $5\text{ cm} \times 5\text{ cm}$ QR codes with high accuracy, and store the extracted information for real-time inventory analysis and management. Key navigation and perception requirements include horizontal positioning accuracy of $\pm 10\text{ cm}$, vertical positioning accuracy of $\pm 2\text{ cm}$, obstacle detection within a 5 m range, path planning time of less than 3 s, and an emergency stop response time of less than 500 ms. The robot must complete a full vertical scan of each rack in under 3 minutes and attain a $> 90\%$ detection rate for $5\text{ cm} \times 5\text{ cm}$ QR codes, while operating on battery power without any external supply during operation.

The mechanical design must prioritize compactness (length $< 600\text{ mm}$, width $< 450\text{ mm}$, weight $< 25\text{ kg}$), a low center of gravity, and anti-tip base geometry to ensure stability during motion and vertical scanning. The system must integrate an emergency stop mechanism, collision avoidance system, and safe maximum speed limits in compliance with warehouse safety standards. Within an indoor warehouse environment featuring smooth, flat floors, standardized stationary racks with visible QR codes, and available WiFi/4G connectivity, the robot should leverage commercial off-the-shelf components and open-source software wherever possible. The final solution must deliver a functional autonomous rover prototype with a tested vertical scanning mechanism, integrated sensor suite and navigation stack, along with complete documentation including system and software architecture, testing and validation reports, user and maintenance manuals, circuit schematics and PCB designs, 3D CAD models and assembly drawings, a detailed bill of materials (BOM), source code, and demonstration videos, culminating in a live hardware demonstration.

1.2 Solution Summary

We employ a compact differential-drive autonomous rover equipped with a 2-stage leadscrew-driven vertical actuation mechanism that carries a high-resolution camera for rack scanning up to 1.8–2 meters. The base navigates the warehouse using a LIDAR-based SLAM map and on-board path planning to position itself accurately in front of each rack, while the leadscrew-driven mechanism performs bottom-to-top (or top-to-bottom) scanning with precise vertical positioning. The perception pipeline processes 1920×1080 images in real time to detect and decode $5\text{ cm} \times 5\text{ cm}$ QR codes, associates them with rack locations, and uploads structured inventory data to the internal storage system which can then be analysed independently later on.

1.3 Key Metrics

- Horizontal positioning accuracy: ± 10 cm
- Vertical positioning accuracy: ± 2 cm
- Obstacle detection range: 5 m
- Path planning time: < 3 s
- Emergency stop response time: < 500 ms
- Maximum rack height: 1.8–2 m
- Vertical scanning time per rack: < 3 min
- Image resolution: $\geq 1920 \times 1080$ pixels
- Scanning repeatability: $> 95\%$
- 5 cm $\times$ 5 cm QR detection success rate: $> 90\%$
- Chassis envelope: length < 600 mm, width < 450 mm
- Total system weight: < 25 kg

1.4 Innovation Highlights

- Integrated horizontal navigation and vertical scanning architecture combining a differential-drive base with a 2-stage leadscrew mechanism for precise rack coverage up to 2 m.
- LIDAR-based autonomous navigation with SLAM and obstacle-aware path planning tailored for structured warehouse aisles and standardized racks.
- High-resolution vision and QR-based inventory identification pipeline optimized for 5 cm $\times$ 5 cm codes under realistic warehouse lighting and motion constraints.
- Safety-first design incorporating low center of gravity, anti-tip geometry, collision avoidance, and hardware emergency stop to meet warehouse safety standards.
- Use of commercial off-the-shelf hardware and open-source software to ensure cost-effectiveness, reproducibility, and ease of maintenance and future upgrades.

Chapter 2

Problem Understanding and Motivation

2.1 Warehouse Challenges

Warehouses designed for human operation pose multiple challenges when automation is introduced. The structure, layout, and working conditions are not optimized for robotic systems, making navigation and scanning difficult.

- **Slow and inaccurate manual scanning:** Manual barcode and QR scanning is time-consuming and often inconsistent, leading to human error and missed data.
- **Physical strain on workers:** Workers are required to bend, reach or climb to access higher shelves, increasing fatigue and risk of injury.
- **Poor real-time inventory awareness:** Inventory data is updated infrequently, causing delays in stock visibility and logistics decisions.
- **Narrow aisles and fixed rack structure:** Warehouse layouts contain tight corridors with limited maneuvering space, making automation difficult.
- **Obstacle-rich environments:** Dynamic and static obstacles such as trolleys, cylinders and personnel increase safety risks for autonomous movement.

Warehouses face serious efficiency and safety limitations due to manual scanning and rigid infrastructure. These limitations create the need for an autonomous system capable of accurate navigation and consistent data collection.

2.2 Current Solution Gaps

Although autonomous robots exist in the industry, many practical limitations prevent their widespread adoption in structured warehouse environments.

- **High system cost:** Commercial AMRs (Autonomous Mobile Robots) rely on proprietary hardware and licensed software, increasing overall system cost.
- **Ineffective vertical scanning:** Most robots lack the ability to scan multiple shelf levels accurately.

- **Low positioning accuracy:** Many robots fail to meet fine navigation requirements required for precise waypoint stopping.
- **Poor navigation in narrow spaces:** Existing robots struggle to operate reliably in narrow aisles due to conservative navigation tuning.
- **Limited safety enforcement:** Few solutions enforce strict safety distance rules around racks and obstacles.

Current automation tools do not adequately address cost, precision, or shelf-level scanning requirements, especially in tight spaces found in warehouses.

2.3 Value Proposition

The proposed robot is designed as a purpose-built autonomous inspection system for warehouse environments.

- **Autonomous navigation:** The system uses SLAM Toolbox for mapping and Nav2 for goal-based movement.
- **Custom safety corridor enforcement:** A keepout filter derived from the map ensures the robot remains within the permitted safety region.
- **Shelf-aware QR scanning:** An optimised scanning motion allows detection of QR codes positioned along vertical rack heights.
- **Low-cost architecture:** Built entirely using open-source software and commercially available hardware.
- **High reliability:** Designed to work consistently in tight and obstacle-rich environments.

The system delivers the precision on par with commercial AMRs at a fraction of the cost by combining smart navigation with shelf-level scanning.

2.4 How We Address These Challenges

The proposed design addresses warehouse automation challenges by integrating mapping, planning, perception, and safety into a unified framework. This enables the robot to autonomously navigate structured environments, effectively manage static and dynamic obstacles, and execute high-resolution vertical inspections with minimal human oversight.

Mapping and Localization

Using SLAM Toolbox, the robot constructs and continuously updates a high-resolution 2D occupancy grid of the warehouse. This mapping solution:

- Provides accurate localization relative to racks and aisles
- Enables repeatable waypoint targeting and autonomous mission execution
- Serves as a shared reference frame for global and local navigation
- Forms the foundation for safety filtering and planning

Through incremental updates, the occupancy grid supports stable localization even after interruptions and ensures navigation remains consistent over repeated inspection cycles.

Navigation with Enforced Safety Corridors

A custom keepout filter is derived from the SLAM map, enabling the robot to navigate within predefined safe zones. This filtering mechanism:

- Establishes buffer margins around racks, walls, and no-go regions
- Guarantees wheel alignment within safe traversal paths
- Prevents accidental rack contact during tight maneuvers
- Forces the planner to respect workspace limits at all times

The filter is incorporated into the planning costmap, ensuring that safety is automatically encoded into the generated trajectories without requiring manual supervision.

Obstacle Handling

Obstacle management is handled using a layered perception approach, combining static representation and dynamic detection:

- Static obstacles (e.g., racks, pillars) are embedded into the keepout filter during mapping
- Dynamic obstacles are detected via LiDAR and inserted into the local costmap in real time
- Motion planning supports reactive avoidance and local replanning during interference

This strategy ensures smooth navigation through aisles while maintaining collision-free motion in environments with both predictable and unpredictable disturbances.

Precise Rack Scanning Strategy

To ensure reliable visual inspection, the robot performs scanning using a dual-position strategy per rack. This approach:

- Enables coverage of the full vertical height without blind spots
- Improves QR detection rate and scanning repeatability
- Reduces motion blur by capturing structured, stable images

By moving between predefined poses rather than relying on a single viewpoint, the robot achieves consistent visual reconstruction of inventory at multiple shelf levels.

System Impact on Warehouse Workflow

The proposed system enhances warehouse operations by automating routine inspection tasks. Benefits include:

- Reduced manual effort for inventory monitoring
- Higher frequency and accuracy of stock updates
- Improved worker safety through reduced physical strain
- Low adoption barriers due to compatibility with existing infrastructure

These improvements enable organizations to implement structured automation without requiring physical modifications to warehouse layouts.

2.5 Competition Objectives

The objective of the competition is to design, implement, and demonstrate an autonomous robotic system capable of navigating a structured warehouse environment and performing reliable rack-level QR scanning while adhering to strict safety and accuracy requirements.

The key technical objectives are:

- **Generation of an accurate map of the warehouse layout** using SLAM techniques.
- **Safe and precise navigation** through narrow aisles using waypoint-based control and obstacle avoidance.
- **Enforcement of safety zones** around racks and walls to ensure the robot remains within the prescribed operational boundary.
- **Reliable QR code scanning** across multiple shelf levels on each rack.
- **High detection accuracy** for small QR codes (5 cm × 5 cm) placed at predefined locations.
- **Efficient task execution** with target completion time under the specified limits for rack scanning.
- **Accurate positioning control** to meet the required horizontal and vertical tolerances.
- **Robust obstacle handling** in the presence of cylindrical static obstacles and dynamic interference.
- **Operator visibility and safety controls** including monitoring interfaces and emergency stop capability.
- **Cost-effective system design** using open-source software and commercially available hardware.

Chapter 3

System Architecture

3.1 Overall Architecture

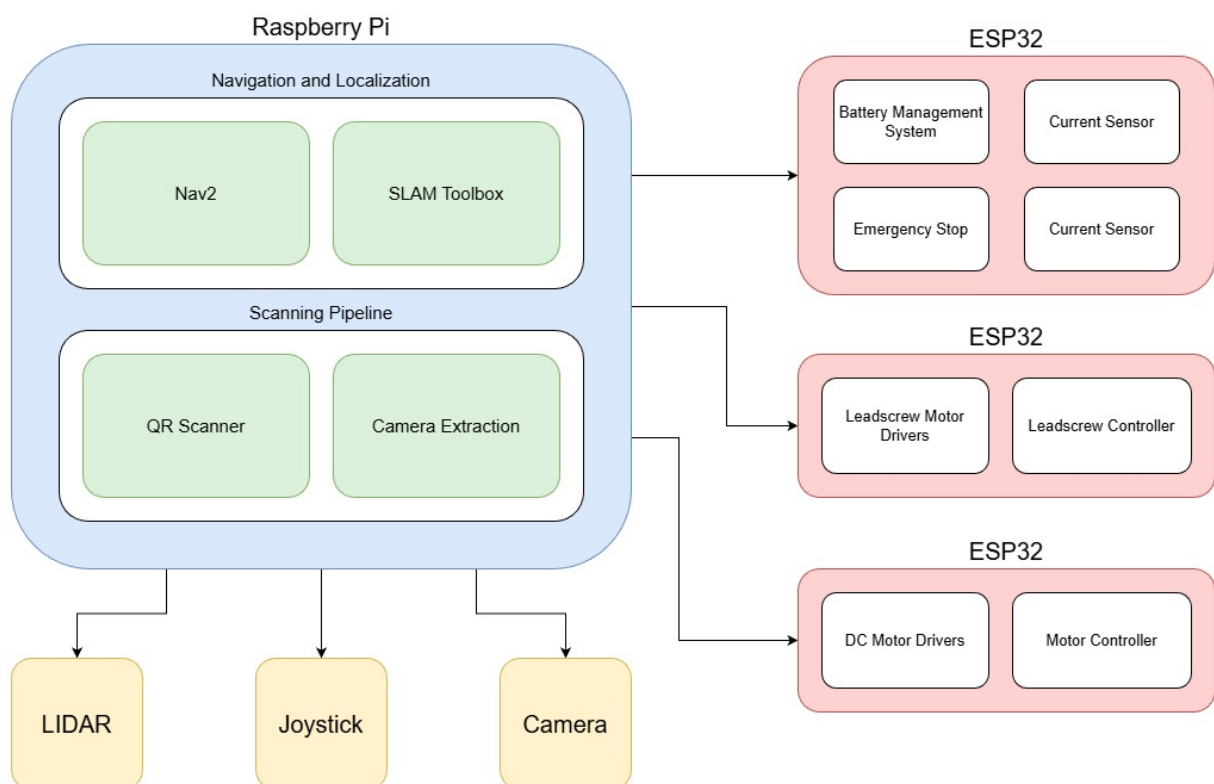


Figure 3.1: Overall Architecture of the Bot

The architecture illustrates the functional and hardware components of the bot. The Raspberry Pi serves as the central processing unit, hosting the software modules: Nav2 and SLAM Toolbox for Navigation and Localization, and QR Scanner and Camera Extraction for the Scanning Pipeline. The Raspberry Pi interfaces directly with the primary sensors: LIDAR, Joystick, and Camera. Peripheral control is handled by three separate ESP32 microcontroller units. One ESP32 manages the Battery Management System, Emergency Stop, and two Current Sensors. A second ESP32 contains the Leadscrew Motor Drivers and the Leadscrew Controller. The third ESP32 includes the DC Motor Drivers and the Motor Controller.

3.2 Hardware Architecture

Raspberry Pi ESP32 Communication

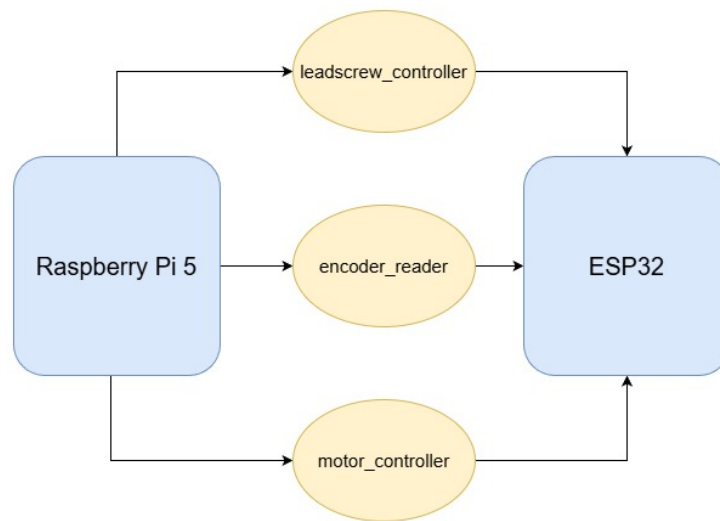


Figure 3.2: Hardware-Software Interface

ROS Nodes Overview

The hardware-software stack is organized around three primary ROS nodes interfacing with embedded controllers: `encoder_reader` for odometry ingestion, `motor_controller` for base actuation, and `leadcrew_controller` for z-mechanism movement.

`encoder_reader` Reads state estimates from the embedded controller and exposes odometry to the ROS ecosystem.

`motor_controller` Consumes velocity commands from ROS and generates motor-level actuation signals for the mobile base.

`leadcrew_controller` Handles vertical actuation of the leadcrew mechanism for rack scanning.

Category	Topic / Interface	Description
Publishes	/odom (nav_msgs/Odometry)	Robot pose estimate (x, y, θ) derived from wheel encoder data.
TF Broadcast	odom $\rightarrow$ base_link	Broadcasts the transform of the robot pose in the TF2 tree for global reference.
Subscribes	None	No subscriptions; operates in a unidirectional data flow from MCU to ROS.
Interfaces / I/O	Serial (ESP32)	Receives 12-byte UART packets containing packed floats (x, y, θ) .

Table 3.1: Node Summary: `encoder_reader`

Category	Topic / Interface	Description
Publishes	None	This node does not publish any topics.
Subscribes	/cmd_vel (geometry_msgs/Twist), /scan_height (std_msgs/Float32)	Desired linear and angular velocity commands to be converted into four mecanum wheel speeds.
Interfaces / I/O	Serial (ESP32)	Sends packed wheel angular velocities over UART for all four motors.

Table 3.2: Node Summary: `motor_controller`

Category	Topic / Interface	Description
Publishes	None	This node does not publish any topics.
Subscribes	/scan_vel (std_msgs/Float32)	Desired vertical velocity command for the leadscrew mechanism during scanning motions.
Interfaces / I/O	Serial (USB $\rightarrow$ Microcontroller)	Sends lead-screw velocity commands over UART at 115200 baud. If no velocity command is received for 0.5 s, a safety stop command "0.0" is sent.

Table 3.3: Node Summary: `leadscrew_controller`

3.3 Software Architecture

Scanning

`slam_toolbox` Subscribes to /scan, /odom and /tf and produces the occupancy grid using AMCL (Adaptive Monte Carlo Localization)/ FastSLAM. The `teleop_controller` Subscribes to joystick commands from /joy and converts it to /cmd_vel, /scan_vel, and also saves the map after warehouse exploration.

Navigation

`goal_setter` Takes the waypoints generated from the processed .pgm file and passes it serially to the BT Navigator for waypoint following.

Category	Topic / Interface	Description
Publishes	/map (nav_msgs/OccupancyGrid)	The generated 2D occupancy grid map built during SLAM.
Subscribes	/scan (sensor_msgs/LaserScan)	Incoming LiDAR laser-scan data.
Subscribes	/odom (nav_msgs/Odometry)	Robot odometry used to estimate incremental motion and improve pose estimation.
Subscribes	/tf, /tf_static (tf2_msgs/TFMessage)	Coordinate frame transformations of the URDF .

Table 3.4: Node Summary: slam_toolbox

Category	Topic / Interface	Description
Publishes	/cmd_vel (geometry_msgs/Twist)	Velocity commands sent to the robot for teleop control during mapping.
Publishes	/start_flag (std_msgs/Bool)	Velocity commands sent to the robot for teleop control during mapping.
Publishes	/scan_vel (std_msgs/Float32)	Velocity commands to the z-mechanism scanner.
Publishes	/map_saver/save_map (nav2_msgs/SaveMap)	Map saver node, creates .pgm and .yaml.
Subscribes	/joy (sensor_msgs/Joy)	Raw joystick input data received from the operators game controller.

Table 3.5: Node Summary: teleop_controller

Category	Topic / Interface	Description
Publishes	/goal_pose (geometry_msgs/PoseStamped)	Publishes the next navigation goal pose to be executed by the Nav2 navigation stack.
Publishes	/scan_flag (std_msgs/Bool)	Indicates whether rack scanning at the current waypoint location has been completed.
Subscribes	/move_flag (std_msgs/Bool)	External trigger used to start or pause autonomous waypoint navigation and rack scanning.
Servers Used	NavigateToPose Action Server	Executes long-distance autonomous navigation to the specified goal pose in the global map.
Servers Used	NavigateToPose Feedback	Provides real-time feedback including navigation progress and remaining distance to the goal.

Table 3.6: Node Summary: goal_setter

3.4 Dataflow Diagram

The following figures illustrate the overall ROS architecture: the first shows the SLAM and mapping components, while the second demonstrates how waypoints are set for the Nav2 navigation system.

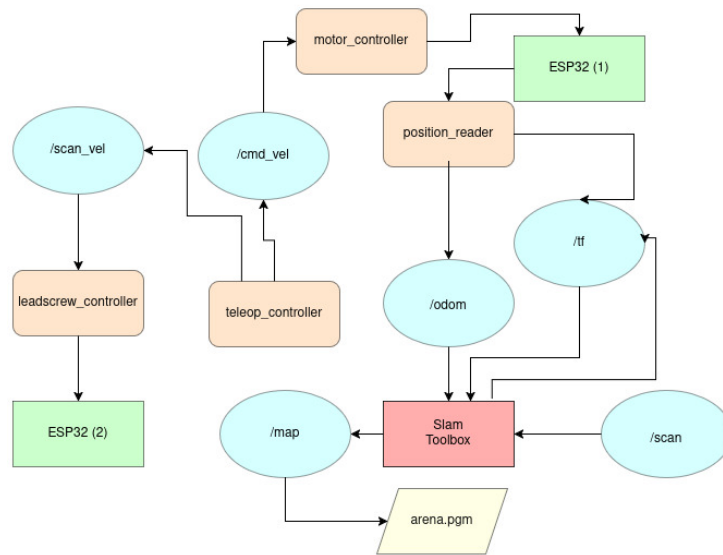


Figure 3.3: Slam ROS Architecture showing the integration of sensors and processing components for simultaneous localization and mapping.

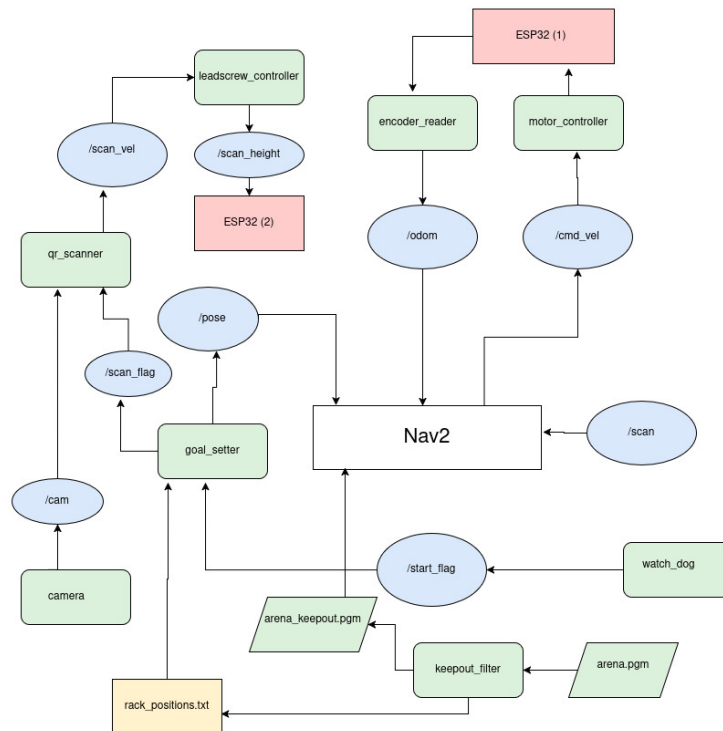


Figure 3.4: Nav2 - Navigation ROS Architecture outlining the path planning and localization components.

Chapter 4

Hardware Design

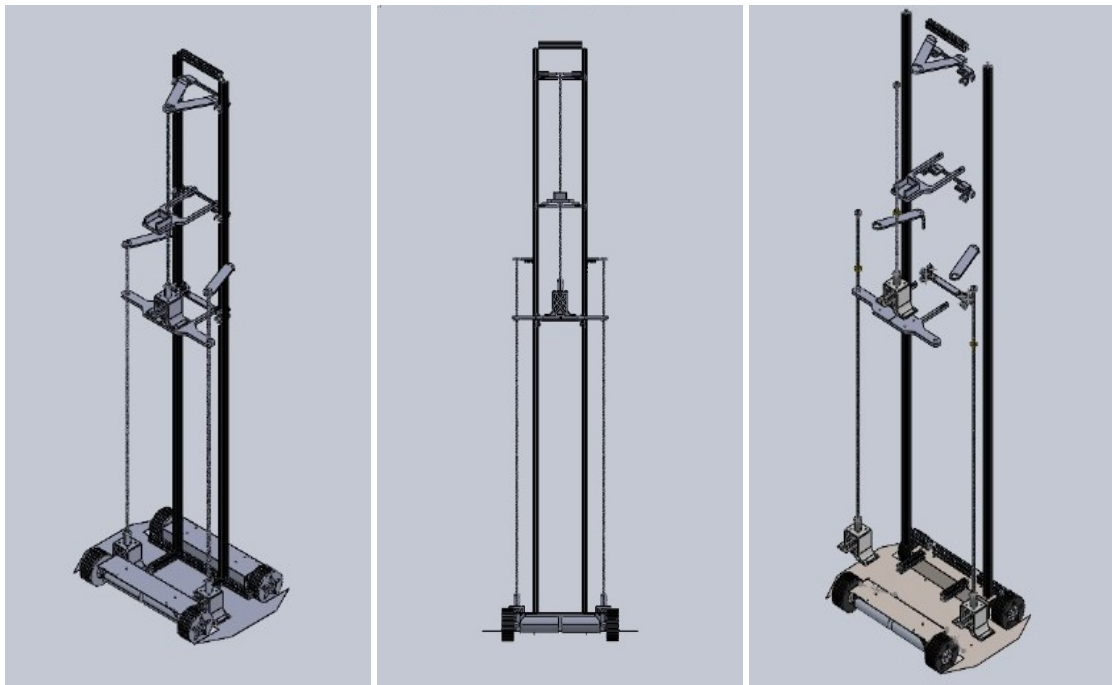


Figure 4.1: Multiple angles of the Dual Stage Leadscrew Bot

4.1 Chassis Specifications

The robot chassis is designed to provide structural stability, compact form factor, and modular mounting space for electronic and mechanical subsystems. The overall dimensions of the chassis are **58 cm in length** and **38 cm in width**, yielding a footprint suitable for maneuvering through standard warehouse aisles. The geometry resembles a “dumbbell” layout, enabling optimal weight distribution while maintaining a low center of gravity.

The base structure is fabricated from a welded steel plate assembly, providing rigidity and load-bearing capacity for the actuation system, lift mechanism, and battery modules. The steel base has a height of **5.12 cm**, and is mounted such that the robot maintains approximately **3 cm of clearance** from the ground when the wheels and motor mounts are attached. This configuration ensures:

- Robust impact resistance in industrial environments,

- Reduced risk of undercarriage interference with floor-level objects,
- Stable ground contact for omni-directional wheel movement.

The chassis is designed with modular mounting points to allow rapid integration and replacement of subsystems during testing and iteration.

4.2 Drive System

The robot employs a **mecanum wheel drive system**, enabling omni-directional mobility for precise aisle alignment and lateral positioning. The drive architecture emphasizes modularity, fault isolation, and maintainability. Electronics are organized into three distinct subsystems, each serving independent and deterministic functions:

1. **Power Regulation Subsystem:** Converts power distribution from the main battery to required voltage levels for embedded controllers, sensors, and peripheral hardware. The system ensures:
 - Stable supply for Raspberry Pi and ESP32 units,
 - Segmented grounding to reduce electrical noise,
 - Overcurrent protection for critical circuits.
2. **Motion Control Subsystem (XY):** Manages omni-directional drive control and trajectory execution. The subsystem communicates velocity commands to four independent wheel actuators, enabling:
 - Holonomic motion control for narrow aisle navigation,
 - Accurate alignment for scanning operations,
 - Modular firmware for rapid reconfiguration.
3. **Vertical Actuation Subsystem (Z):** Implements closed-loop control of the dual-stage lift mechanism. The subsystem is designed around:
 - Dedicated microcontroller for high-frequency command execution,
 - Safety interlocks for overtravel prevention,
 - Plug-and-play connectors for maintenance and debugging.

Pin selection, connector layout, and PCB routing were designed to optimize **solderability, cable access, and modular disassembly**. Power distribution boards allow rapid subsystem isolation during testing.

4.3 Vertical Lift Mechanism

The vertical scanning system consists of a **dual-stage leadscrew assembly** designed to translate the sensing platform along the Z-axis with high accuracy and minimal vibration. The mechanism provides:

- Smooth vertical translation over the full rack height,
- High positional repeatability for image acquisition,
- Structural rigidity under dynamic motion loads.

To minimize mechanical jitter and lateral wobble, multiple support braces and guides are positioned along the leadscrew assembly and sensing platform. The maximum lift speed is approximately **1 cm/s**, which enables full traversal of the scanning range in roughly 10 seconds. This speed profile is sufficient for QR acquisition tasks while maintaining image clarity and motion stability.

4.4 Sensor Suite and Justification

The sensor architecture supports reliable localization, scan integrity, and fault recovery. It integrates both environmental perception and system-state awareness:

- **Limit Switches:** Installed at mechanical endpoints of the Z-axis to:
 - Establish global reference after resets or sudden power loss,
 - Prevent mechanical overtravel and motor burnout,
 - Support homing cycles during initialization.
- **Lidar:** Used for mapping and real-time localization within the warehouse. The lidar input enables:
 - Generation of a 2D occupancy map for path planning,
 - Consistent obstacle detection and avoidance,
 - Safe navigation in dynamic environments with minimal operator supervision.

Together, the sensors ensure that the robot maintains spatial awareness, operational safety, and environmental adaptability during autonomous inspection tasks.

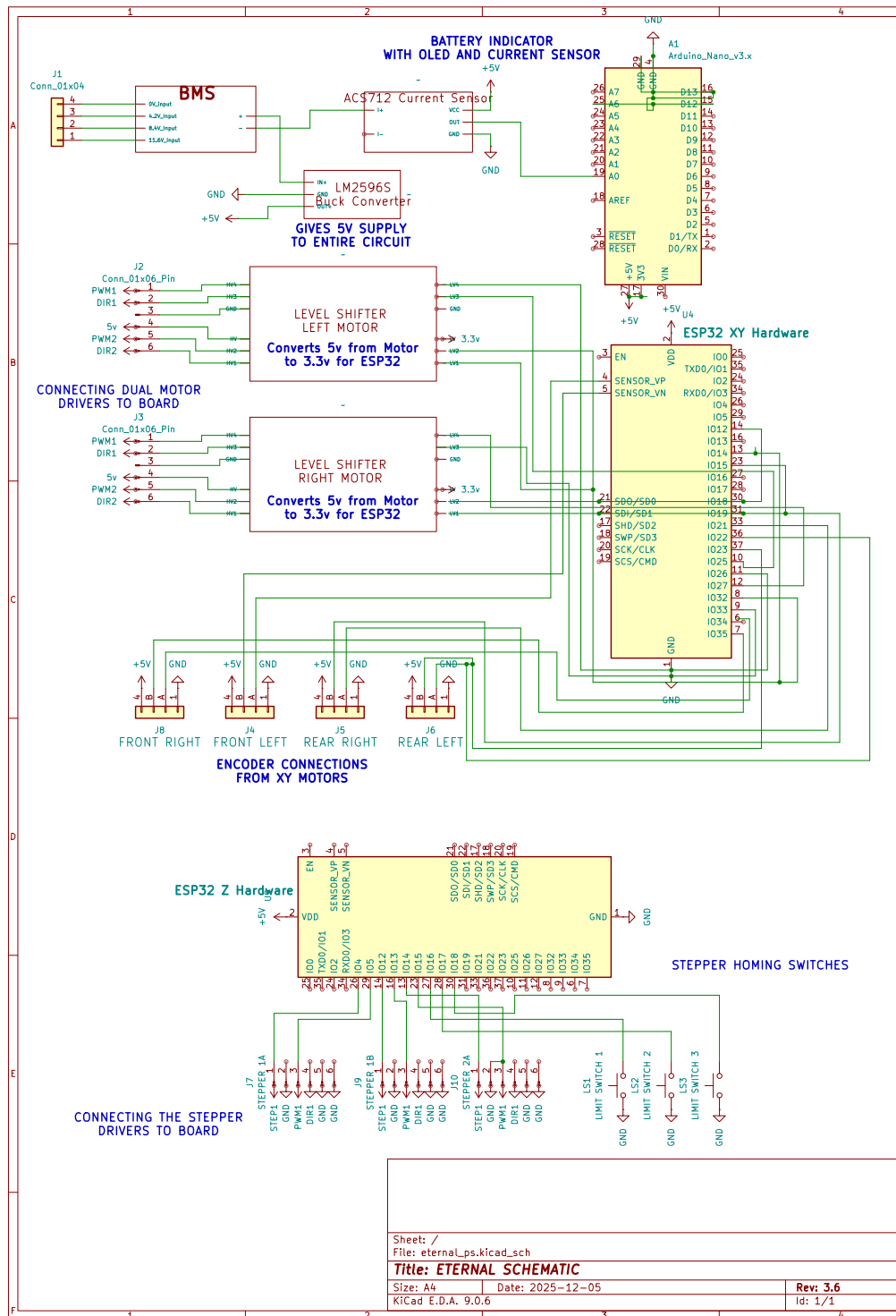


Figure 4.2: Electrical Schematic of Bot

Chapter 5

Software Architecture

5.1 Scanning Pipeline

Objective

The mapping module necessitates an initial initialization phase wherein the robot is navigated via tele-operation. During this manual control period, the robot aggregates sensor data to synthesize a global occupancy grid map that accurately reflects the static layout of the warehouse.

Procedure

1. Launch `slam_toolbox` in online mapping node.
2. Drive robot manually through aisles using **Teleop Key**.
3. Save map as a `.pgm` image which is basically the occupancy grid image and the map parameters in a YAML metadata file.

Map Generation

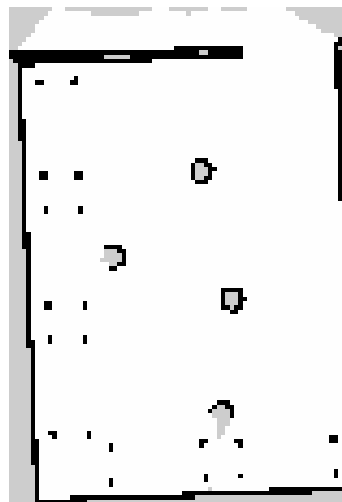


Figure 5.1: Warehouse occupancy grid

High-definition mapping of environmental features was achieved by executing a thorough, multi-angle traversal of the operational domain, ensuring sharp definition of shelf and obstacle boundaries.

Keepout Filter Generation

Purpose

The keepout filter represents zones that the robot must not enter. It embeds safety knowledge directly into the navigation stack, reinforced with the existing local and global costmaps of Nav2 Stack.

.pgm Processing Pipeline Summary

```
Read arena.pgm + YAML metadata
Detect corners -> perspective transform
Enhance + threshold rectified map
Mask structural regions (top/bottom/side black strips)
Find small contours (rack legs)
Cluster -> compute rack horizontal/vertical legs
Compute optimal rack docking positions
Draw safe region around obstacles
Draw black keepout corridors + white guide stripes
Adjust docking positions to nearest free (white) regions
Undo perspective transform -> original map frame
Convert pixel coords -> map coords
Overlay keepout filter on original SLAM map
Save keepout filter + write pose file
```

Processing Method

Load map and YAML metadata:

Load the .pgm and extract origin and resolution from the .yaml file.

Perspective transform:

Detect rectangle corners using contour analysis and Shi-Tomasi corner detection, compute the minimum-area bounding box, and apply a perspective transform to crop to the part of the map which we actually need (the area within the walls). A perspective transform maps a point (x, y) to (x', y') using a 3×3 homography matrix H :

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}.$$

The final pixel coordinates are obtained by normalization:

$$x'' = \frac{x'}{w'}, \quad y'' = \frac{y'}{w'}.$$

Initial preprocessing:

Invert, dilate, and threshold the rectified grayscale map to enhance rack legs and isolate structural geometry.

Rack Detection:

- Extract small isolated components (rack legs) using contour area filtering.
- Compute the geometric centroids of each rack leg.
- Categorize them into horizontal and vertical rack groups.

Region masking:

Remove top, bottom, and side strips corresponding to the start of the safe region which is obtained by giving a 10cm offset from the walls and the racks. Make these regions black in the filter pgm.

Marking the safe zone:

Make a strip of white color of width 15cm from the currently masked zones towards the inside of the arena essentially making a 15cm wide zone for moving around the entire arena.

Obstacle detection, inflation and structuring:

- Find the min-enclosing circle around each obstacle in the cropped region.
- Define the safe zone for the obstacle as 15cm to 30cm from the edge of the obstacle.
- Make a smaller inner keepout zone of radius 25cm from the center of obstacle.
- Also make a larger outer free zone, with same center and radius 40cm.
- Do the region masking again to prevent free zones of obstacles from crossing the safe offsets of walls or racks.

thereby creating a black-on-white representation of safe regions around obstacles.

Rack pole-based optimal waypoint detection:

- Estimate rack-level optimal docking positions(for stopping and scanning the QR codes) based on rack geometry. (optimal way-points)
- Sort the optimal waypoints such that the bot moves in a clockwise direction with two waypoints (at $\frac{1}{4}^{th}$ and $\frac{3}{4}^{th}$ distance) between each rack, totalling 10 waypoints.

Adaptive correction of optimal way-points for scanning:

All optimal rack approach points are shifted inward until their local neighborhood is guaranteed to be a free-zone.

Reverse perspective transform:

Warp the generated mask back into the original SLAM-map coordinate frame, using the inverse of the transformation matrix obtained using the perspective transform.

Coordinate conversion:

Convert pixel coordinates to global map coordinates using:

$$x = x_0 + u \cdot resolution, \quad y = y_0 + (H - v) \cdot resolution.$$

Export keepout filter and optimal waypoints:

The resulting mask is saved as a white-on-black image and visualized over the original map using alpha blending and the global map coordinates are written to a .txt file for waypoint navigator node.

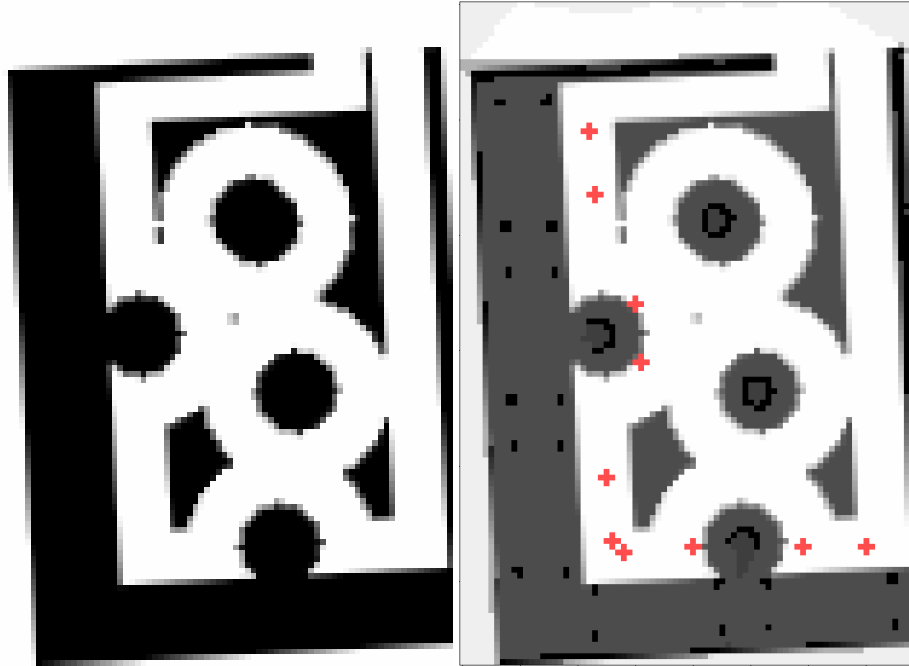


Figure 5.2: Generated keepout filter, with safe regions (white) and lethal cost regions (Left) and Overlay filter with optimal waypoint coordinates (Right)

5.2 Navigation Pipeline

Overview of the Navigation2 Stack

The Nav2 stack operates by converting the occupancy grid map into cost representations and performing path planning within this cost-based representation. The robot navigation is implemented using the ROS2 Navigation2 (Nav2) stack, which provides modular components for localization, path planning, control, and obstacle avoidance. The main subsystems involved in Nav2 are:

- Global planner (for generating long-term paths), Local controller (for real-time motion execution), Global costmap (for global obstacle awareness), Local costmap (for short-range collision avoidance), Behavior tree (for sequencing navigation tasks).
- The keepout map (.yaml) is subscribed by the `costmap_filter_info_server`
- The parameters of the Navigation2 stack can be tuned with the launch parameter file for Nav2 in the `/config` folder of `warehouse_navigation`.
- The `nav2lifecycle.sh` script provides an automated sequence of lifecycle transitions by activating each navigation component individually.

- Entering the keepout zone incurs a 'lethal_cost', which prevents the `bt_navigator` from planning trajectories which coincide with keepout zones.
- The `goal_setter` sends goals to Nav2 BT Navigator through the `BasicNavigator()`.

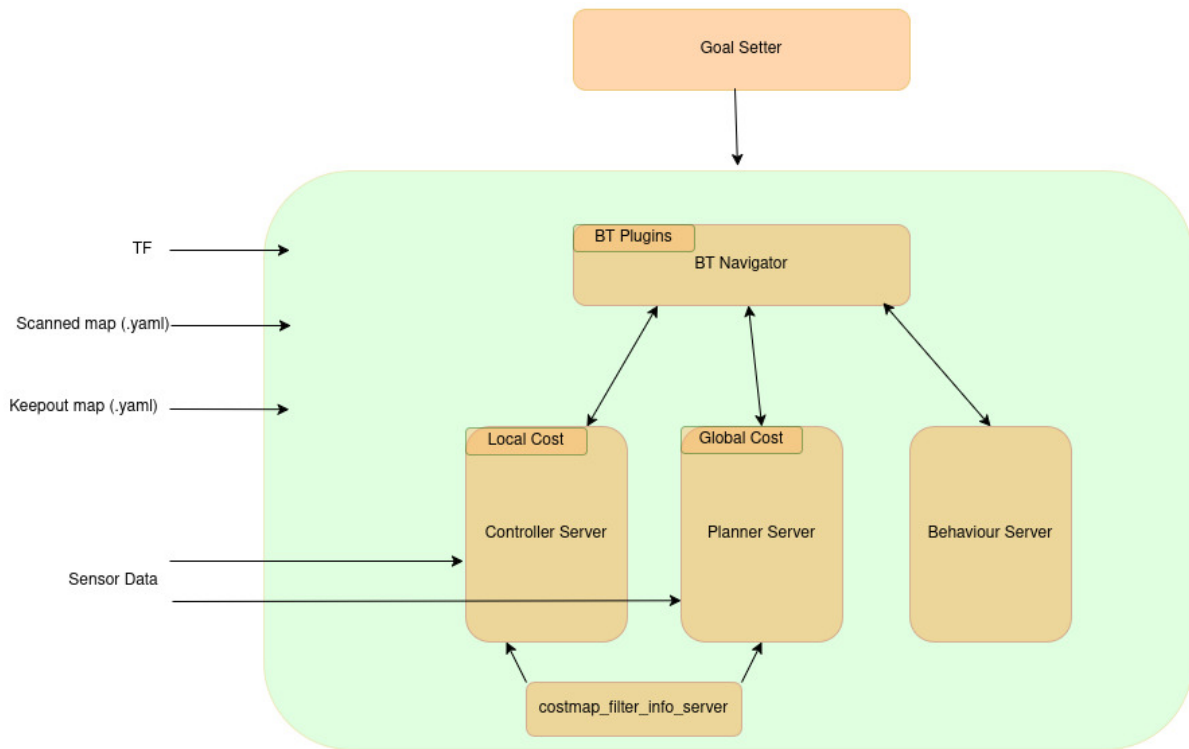


Figure 5.3: Nav2 Architecture

Keepout Filter as a Costmap Constraint

To navigate within preferred regions/zones we add a keepout filter from the `.pgm` processing pipeline to specify keepout zones to Nav2. The custom-generated keepout filter is integrated into the Nav2 costmap as an additional static layer. This layer imposes artificial obstacles that encode predefined safety constraints into the navigation system. The keepout filter image consists of:

- **Black regions**, forbidden zones (outside safety corridor),
- **White corridors**, permitted robot motion areas, narrow navigation lanes near racks.
- **Black circles**, obstacle keepout zones around cylindrical obstacles.
- Its implemented as a `Costmap2D Filter Plugin`, and operates inside both global costmap and local costmap

This filter is converted to an occupancy grid and added to the global and local costmaps. Cells marked as black are assigned lethal cost values, preventing planners from generating paths through those regions. Figures below show the keepout filter applied as an overlay on the Nav2 costmaps.

The overlay ensures that, the robot cannot physically enter restricted regions, the navigation engine naturally selects paths aligned with the safety corridor and obstacle safety margins are respected through artificial expansion zones.

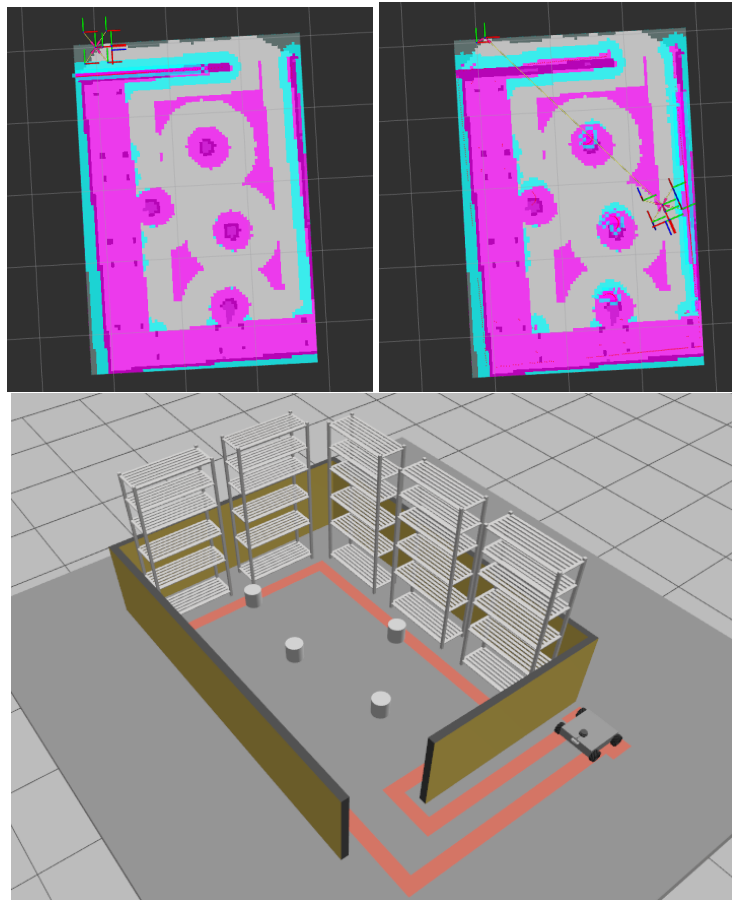


Figure 5.4: Global costmap with keepout filter overlay

Global Path Planning

The Planner Server in Nav2 is responsible for generating a global path from the robot's current pose to a specified goal pose. It does not command velocities; instead, it computes a static path that the Controller Server later follows.

The planner server loads a Grid-based planner, `NavfnPlanner`, which uses the A* algorithm to compute global paths. The global costmap encodes static obstacles, keepout regions, and map boundaries. The planner treats this costmap as a 2D grid, where each cell has a traversal cost.

The planner selects the lowest-cost trajectory (using Nav2), where it:

- Receives a goal pose
- Builds the planning graph from the global costmap
- Runs `NavFn` default, generates a feasible path.
- Sends the path to `bt_navigator`

This enables the robot to follow rack geometry almost like a virtual guide rail.

Local Path Planner

While the global planner generates a coarse path, the local planner continuously updates and computes velocity commands at a higher frequency, focusing on short-horizon, reactive motion using live sensor

data. The local planner generates safe velocity commands (v, ω) that prevent collision with inflated obstacles, keep the robot inside the permitted corridor, and avoid oscillation near cost boundaries,

Wheel Safety Constraint Enforcement

A critical constraint is that one side of robot wheels must always remain inside the safety corridor. This is implemented indirectly by shaping the corridor width to match wheel track, positioning lethal boundaries exactly at corridor edges and tuning inflation so only valid wheel-projected poses remain feasible. This causes invalid postures to be rejected automatically during planning, and also implements **dynamic** obstacle avoidance.

5.3 QR Code Acquisition

The pipeline automates item identification and logging from moving warehouse racks by integrating motion, actuation, and real-time decoding.

1. Controlled Mobile Base Motion:

- Robot maintains a constant forward velocity for sharp images and reliable QR detection.
- Enables continuous scanning without frequent stops or pauses.

2. Autonomous Vertical Scanning:

- Dedicated Z-axis leadscrew mechanism autonomously moves the camera across rack height.
- Executes up/down motions based on predefined trajectories for complete shelf coverage.
- Vertical routine starts after base alignment, ensuring consistent field-of-view overlap.

3. Real-time QR Decoding and Storage:

- Computer vision node performs continuous real-time decoding of captured frames.
- Extracted contents (product ID, location) are stored locally on the Raspberry Pi.
- Data is saved in a structured format (CSV/JSON for easy integration.)

4. Operational State Monitoring and Fault Handling:

- System logs all key operational states (pose, detections, actuator status).
- Runtime checks identify errors (missed detections, mechanical faults).
- Appropriate fault flags are raised to trigger safety routines or scan retries.

Overall, the QR acquisition subsystem provides a robust and autonomous pipeline for structured rack scanning, enabling accurate inventory capture without manual intervention, while ensuring traceability, safety, and data integrity throughout the process.

5.4 HMI Design

The Human Machine Interface provides:

- Live map display (SLAM map + robot pose) through Rviz2.

The HMI communicates with ROS 2 topics and services, enabling monitoring and intervention when required.

5.5 Communication Protocols

The communication backbone relies on ROS 2 DDS middleware. Major topics include:

ROS2 Topics and Data Flow

The following ROS2 topics form the communication backbone of the warehouse robot system. They provide real-time data exchange between the SLAM module, Nav2 navigation stack, perception nodes, and the HumanMachine Interface (HMI).

- `/scan`

Message Type: `sensor_msgs/msg/LaserScan`

This topic publishes LiDAR range measurements at regular intervals. Each message consists of an array of distance values corresponding to angular measurements around the robot along with parameters such as angle increment, minimum range, and maximum range.

Usage:

- Used by SLAM Toolbox to build the occupancy grid map during Round 1.
- Used by Nav2 local costmap and navigation controller for dynamic obstacle inflation, collision checking, and real-time re-planning during Round 2.

- `/cmd_vel`

Message Type: `geometry_msgs/msg/Twist`

This topic handles linear and angular velocity commands for robot motion control. The message consists of:

- Linear velocity (x, y, z) in meters per second.
- Angular velocity (roll, pitch, yaw) in radians per second.

Usage:

- During SLAM mode, teleoperation nodes publish velocity commands to allow manual driving.
- During navigation mode, Nav2 publishes autonomous velocity commands based on path following.
- The robot motor controller subscribes to this topic to execute movement.

- `/odom`

Message Type: `nav_msgs/msg/Odometry`

Description: This topic contains incremental odometry data including robot position, orientation, and velocity estimates derived from wheel encoders.

Fields include:

- Pose: position (x, y, z) and orientation (quaternion).
- Twist: linear and angular velocity estimates.

Usage:

- Used by SLAM Toolbox to refine pose estimation.
- Used by Nav2 for feedback control and localization.
- Used by the TF system to broadcast relative motion between frames.

- /tf

Message Type: `tf2_msgs/msg/TFMessage`

Description: This topic maintains the real-time transform tree describing spatial relationships among coordinate frames. The following transforms are maintained:

- `map odom`: Provided by SLAM Toolbox or AMCL.
- `odom base_link`: Provided by the robot odometry node.

Usage:

- Used by Nav2 to compute robot pose in the global map.
- Used by RViz and HMI for visualization.
- Used by perception nodes to associate sensor data with map coordinates.

- /cam

Message Type: `sensor_msgs/msg/Image`

Description: Provides raw camera frames captured by the onboard camera. Frames are published in bgr8 encoding by the `camera_node`.

- Contains pixel data for each captured frame.
- Includes a header with timestamp and frame ID.
- Encoding format: bgr8.

Usage:

- Published by the `camera_node` at ~ 30 FPS.
- Subscribed to by the `qr_scanner` node for QR detection.
- Supports all vision-based tasks requiring live camera input.

- /goal_pose

Message Type: `geometry_msgs/msg/PoseStamped`

Description: Provides target waypoint positions for navigation. Each message contains:

- Position (x, y, z).
- Orientation as quaternion.
- Frame reference (usually `map`).

Usage:

- Generated by the `waypoint_planner` node.
- Received by Nav2 behavior tree to initiate navigation.
- Allows the robot to move between predefined rack scanning positions.

Services such as `NavigateToPose`, and `Lifecycle` management ensure reliable sequential execution.

Additionally, **UART communication** is used between the Raspberry Pi and the ESP32, where data is exchanged as simple byte streams over TX/RX lines. This provides a lightweight and reliable channel for transmitting low-level commands and sensor feedback.

Chapter 6

Integration and Control

6.1 Hardware Software Interaction

The interaction between hardware and software is designed to provide a reliable interface between high-level autonomy modules and low-level embedded control hardware. The **Raspberry Pi** functions as the central computing platform, running ROS for navigation, perception, and system orchestration. Low-level motor actuation, including both the drive system and vertical lift mechanism, is handled by **ESP32 microcontrollers**, which execute deterministic control loops.

Communication between the Raspberry Pi and ESP32 units is implemented via a **UART serial link** using a structured, predefined message format. The Raspberry Pi publishes velocity and actuator commands as ROS messages, which are serialized and dispatched to the ESP32 controllers in real time. Feedback packets containing odometry and actuator state are transmitted back to ROS for localization, error handling, and state monitoring. This architecture ensures:

- Low-latency command execution for motor control,
- Modular software deployment without firmware modification,
- Clear separation between high-rate control loops and navigation algorithms.

In addition to autonomous actuation, a **joystick interface** is incorporated to allow manual teleoperation during the initial map-building phase. Teleoperation provides:

- Human-supervised control in uncharted environments,
- Fast dataset collection for SLAM initialization,
- A safe fallback mode during tuning and testing.

The joystick inputs are translated into standard `/cmd_vel` ROS messages, allowing seamless integration with existing motor control nodes.

6.2 Fail-Safe Mechanisms

Robust failsafe mechanisms are integrated at both the hardware and software level to ensure safe operation in warehouse environments. A **physical emergency stop button** is installed on the chassis, which instantly cuts power to the entire drive and actuator circuitry when activated. This safety measure prevents uncontrolled motion and protects hardware and personnel in the event of system malfunction.

To prevent mechanical damage, the robot incorporates **mechanical hard stops** on critical moving components, such as the leadscrew-driven lift mechanism. These stops prevent overextension and protect against structural stress caused by sensor failure or unexpected command sequences.

Environmental awareness and collision avoidance are provided by the **LIDAR-based navigation system**. Real-time perception and local costmaps enable:

- Obstacle detection within the robots operational envelope,
- Dynamic replanning in cluttered or changing environments,
- Safe navigation without user intervention.

Software-level failsafes complement hardware protections. These include:

- Virtual emergency stop routines triggered through ROS services,
- Watchdog timers on the microcontrollers to halt motion if commands are lost,
- Safety flags raised when invalid sensor data or actuator errors are detected.

The combination of hardware and software safety systems ensures that the robot remains **operationally safe, fault tolerant, and compliant with warehouse safety standards**. In the event of abnormal behavior, redundant safety layers guarantee that motion is halted before damage can occur, protecting both the robot and its surroundings.

Chapter 7

Testing and Validation Summary

7.1 Methodology Overview

To evaluate system reliability and performance under realistic warehouse operating conditions, a structured testing methodology was developed for both the perception subsystem and motion control architecture. The **QR code scanning subsystem** was validated by placing encoded tags at various positions along a simulated rack environment and altering key environmental variables such as viewing angle, ambient lighting, and camera-to-object distance. Each test case was repeated multiple times to assess detection consistency and robustness. The decoded data was compared against the ground truth string to evaluate correctness, and failure cases were logged to determine whether errors stemmed from optical limitations, motion blur, or environmental distortion.

The **XY motion subsystem** was validated by commanding the robot to traverse a predefined square trajectory using pure odometry feedback. Following loop completion, the final position was compared against the starting pose to determine accumulated drift. Metrics such as translational offset, angular deviation, and path smoothness were recorded. Multiple trials were conducted at varying velocities to capture the influence of motion speed on control accuracy. These tests provide insight into both intrinsic mechanical precision and the performance of the localization pipeline, particularly under repetitive motion patterns. Data from each trial was logged, aggregated, and statistically analyzed to quantify performance trends and identify potential causes of systematic error.

7.2 Key Performance Metrics

Evaluation of the robots performance focused on quantifiable metrics that capture the reliability, consistency, and functional suitability of the system for warehouse operations. The primary metrics include:

- **Accuracy:** Measures how closely the system's outputs (e.g., estimated pose, QR decoded values) match ground truth. For navigation, accuracy is represented as positional error (in cm) and angular error (in degrees). For the scanning pipeline, accuracy captures successful character-level decoding of QR content. High accuracy is critical for maintaining correct spatial awareness during autonomous operation.
- **Repeatability:** Assesses the ability of the system to consistently reproduce results across repeated trials under identical conditions. For motion control, this is measured as the variance of final pose error across multiple runs of the same trajectory. For QR scanning, repeatability captures the percentage of successful detections across repeated exposures. High repeatability indicates stable performance unaffected by noise, drift, or environmental factors.

Additional qualitative metrics, such as system robustness under poor lighting, sensitivity to lens distortion, and resilience to encoder noise, were also evaluated during testing. Collectively, these metrics helped verify that the robot meets functional requirements for inventory scanning, autonomous navigation, and safe operation in structured warehouse environments.

7.3 Test Results Summary

Overall, the system demonstrated performance that closely aligns with the intended operational specifications, validating the feasibility of the proposed design within warehouse environments. The QR code scanning subsystem consistently achieved high detection accuracy across a range of lighting, orientation, and distance conditions, with only minor failures occurring under extreme viewing angles or severe glare. These errors were infrequent and did not significantly impact data extraction reliability, indicating that the optical pipeline is sufficiently robust for real-world deployment.

The motion control subsystem achieved near-spec performance in terms of positional accuracy and repeatability. During square-path trials, the robot reliably completed the trajectory with low cumulative drift, and final pose deviations were within acceptable margins for autonomous aisle traversal and rack alignment. Repeatability tests further confirmed that the system produced stable results with low variance across multiple iterations, demonstrating consistent holonomic behavior and effective actuator response. While minor positional offsets were observed at higher velocities, they remained within tolerable limits and did not compromise task execution.

Collectively, the test results confirm that the integrated hardware software architecture performs reliably under operationally relevant conditions, meeting core requirements for navigation precision, scanning quality, and safety. The system exhibits strong baseline performance and provides a solid foundation for further optimization, scaling, and deployment in structured warehouse environments.

Chapter 8

Innovation and Novel Contributions

8.1 Unique Design Choices

Employs a hybrid modular-mechanical architecture that separates horizontal navigation from vertical actuation, allowing each subsystem to be optimized for its own functional requirements without introducing cross-coupled complexity. Unlike conventional designs where sensing hardware is mounted directly onto the drive chassis, this robot uses a dual-stage, mechanically isolated vertical lift column that minimizes vibration propagation from the drive system to the camera platform. This mechanical decoupling enables sharper images during motion and eliminates the need for computationally expensive motion-blur compensation techniques. Additionally, the chassis adopts a distributed mounting strategy where electronics, batteries, and actuation hardware are positioned strategically to maintain balance across varying payload distributions, enhancing stability and reducing control overhead during rapid maneuvers.

Another unique design choice is the emphasis on low-cost extensibility while preserving industrial reliability. Rather than integrating all functionality into a single monolithic board, the system uses a network of microcontroller-driven subsystems, each handling deterministic tasks such as mecanum actuation, vertical lift control, or safety-critical monitoring. This decentralization reduces the risk of single-point failure and allows independent upgrading or scaling of components. Power and data interconnects are standardized across subsystems to support plug-and-play maintenance, enabling rapid field repairs without specialized equipment. Combined with open-source software integration and a modular mechanical platform, these choices result in a system that is highly configurable, easy to maintain, and economically viable for large-scale deployment in warehouse automation environments.

8.2 Algorithm Optimizations

After generating the map using SLAM Toolbox, a black-and-white keep-out layer is added, where black regions represent forbidden areas and white regions represent permitted traversal corridors. Instead of providing Nav2 with arbitrary waypoints, the system constrains all waypoints to lie strictly within the white regions. This procedure constitutes an algorithmic optimization for the following reasons:

Reduction of the Planner's Search Space

Let the global planner operate over a grid map represented as a set of states S . Introducing a keepout mask removes a large subset S_{black} from the feasible state space, leaving the planner with a restricted domain

$$S' = S \setminus S_{\text{black}}.$$

This reduction in search space decreases the number of node expansions required by graph-based planners (A*, Smac, NavFn), which directly lowers computational complexity. This is a classical form of algorithmic optimization: pruning the search domain to reduce planning time.

Transformation of Global Planning into Constrained Path Following

Without the `keepout` layer, the planner must discover collision-free navigation paths through arbitrary free space. The white regions effectively encode a predefined subgraph G_{white} of admissible corridors. As a result, the planning problem changes from:

Find a path in S , to

Follow a path within G_{white} ,

which is a substantially simpler problem. This converts global path planning into constrained graph traversal, reducing the planner's branching factor and improving its efficiency.

Offloading Complexity Through Preprocessing

Adding the keepout layer is a preprocessing step that imposes structure on the environment before online planning begins. This shifts computational burden from the runtime planner to the offline map-design phase, enabling:

- simplified online planning,
- a more structured and predictable state space,
- reduced variability in generated paths.

Preprocessing for structural simplification is a well-known algorithmic optimization technique.

Increased Predictability and Reduced Entropy

By constraining the robot to move solely through the white corridors, the range of possible trajectories is dramatically reduced. This lowers the entropy of the planner's decision space and yields more repeatable, predictable navigation behavior.

Thus, the method represents a clear and practical algorithmic optimization.

8.3 Cost and Efficiency Innovations

The current material selection (steel, aluminum, PLA, and HK camera) offers immediate prototyping benefits but requires optimization for scalable cost efficiency and long-term durability.

Steel Base (Chassis)

- **Cost Efficient:** Steel is highly cost-effective and provides necessary rigidity and load-bearing capacity.

- **Innovation Focus:** Explore specialized high-strength steel or structural aluminum alloys to reduce weight and improve energy efficiency, cutting long-term operational costs.

Aluminum Guard Rails

- **Cost Efficient:** Aluminum provides excellent lightweight, corrosion resistance, and moderate cost for protective structures.
- **Innovation Focus:** Standardize aluminum profiles to simplify manufacturing and assembly, further reducing per-unit production complexity and cost.

PLA Plastic (3D Printed Material)

- **Cost Efficient:** PLA is a low-cost, easily processed material ideal for rapid, iterative design and minimizing initial tooling costs.
- **Innovation Focus:** Transition volume production to Injection Molding using industrial polymers (e.g., ABS/Nylon) for superior durability and significant per-unit savings at high scale.

HK Camera

- **Cost Efficient:** The HK camera provides an acceptable balance of performance and moderate unit cost for the prototype stage.
- **Innovation Focus:** Conduct a Total Cost of Ownership (TCO) analysis to select a more widely available, lower – cost OEM module. Alternatively, justify a higher-cost camera if it substantially increases first-pass scan success rates, improving operational efficiency.

Chapter 9

Limitations and Future Scope

9.1 Known Limitations

The current system, despite its successful core functions (mapping, safe navigation, rack scanning), faces several specific technical and design constraints that hinder its reliability and potential for wider deployment.

- **Dependence on 2D LIDAR:**

- The core obstacle detection system relies exclusively on a **2D planar LIDAR scan**, which provides data only for a single horizontal slice of the environment.
- **Constraint Impact:** Obstacle detection is therefore limited to the plane of the scanner. The system is blind to obstacles that are significantly low - lying (e.g., small pallet jacks or fallen debris) or overhanging (e.g., items protruding from a high rack or overhead pipes) because they do not intersect the 2D scan plane. This significantly compromises safety and robustness in dynamic warehouse environments.

- **Static Map Assumption:**

- The navigation stack (likely based on SLAM) fundamentally operates under a static environment assumption. It depends on a pre-generated map and a filtered keepout map created and assumed not to change during operation.
- **Constraint Impact:** The system cannot automatically adapt to significant or sudden dynamic rearrangements in the warehouse layout (e.g., shifting racks or temporary barriers). If a section is blocked, the bot may attempt to navigate through it, leading to deadlocks or requiring manual intervention to update the map, which reduces operational autonomy.

- **Optimal Pose Generation Has Geometric Constraints:**

- While the computation of the optimal approach rack pose (position and orientation for successful scanning) is automated, this process is based on geometric analysis of the environment map. It requires clean segmentation and the extraction of accurate geometry (e.g., straight lines) from the map data.
- **Constraint Impact:** Map inaccuracies (e.g., drift, noisy sensor data), poorly segmented rack data, or racks that are not perfectly aligned/orthogonal (common in older facilities) can lead to the generation of a sub-optimal or impossible approach pose. This results in failed rack scans or the bot getting too close, reducing the reliability of the core scanning function.

- **Additional Waypoints Near Corners for Smoother Movement:**

- The navigation path planning requires manual insertion of additional waypoints placed strategically near corners or sharp turns in the path.
 - **Constraint Impact:** This necessity indicates that the underlying path planning algorithm or local trajectory planner is not inherently robust enough to generate smooth, collision-free paths for sharp turns without external tuning. Relying on manually-placed waypoints is not scalable and error-prone, especially when dealing with high-speed movements, necessitating a safety buffer.
- **Recovery Under Tight Spaces:**
 - The robot utilizes the default **recovery behaviors** provided by the **Nav2** framework (e.g., turning in place, driving backward).
 - **Constraint Impact:** In environments with tight constraints such as narrow aisles, these standard recovery maneuvers are often ineffective or dangerous. The recovery behavior may cause the robot to collide with another wall or obstacle while attempting to clear the blockage, leading to a complete system failure or damage.
- **Action Feedback Latency:**
 - The critical emergency stop capability is intrinsically linked to the **Action Feedback Rate** of the `NavigateToPose` action.
 - **Constraint Impact:** The time delay between a required stop event (e.g., a sudden obstacle appearing) and the system receiving the signal to initiate the stop introduces a latency. This slight delay becomes critical during high-speed navigation, potentially increasing the stopping distance beyond the calculated safety margin and risking a collision.

9.2 Commercialization Path

To achieve market readiness and overcome existing prototype constraints, development must focus on these four pillars:

Production Grade Software and Autonomy

Focus on reliability, latency reduction, and robust failure handling.

- **Robust RTOS Implementation:** Use a deterministic Real-Time Operating System for safety-critical functions to eliminate **Action Feedback Latency**.
- **Context-Aware Recovery:** Implement specialized failure recovery behaviors for narrow aisles to resolve issues in **Tight Spaces**.
- **Testing and Standards:** Adhere to industrial software standards and comprehensive testing to achieve **Production Grade Software**.

Industrialized Hardware Design

Focus on durability, consistency, and manufacturing scalability.

- **Custom PCB Adoption:** Use **PCB for Circuitry** to provide reliable, fixed electrical connections and improve **robustness**.

- **Material Upgrade:** Utilize industrial-grade materials for the chassis to improve longevity and resilience (**Improvements in Materials Used**).
- **Component Sourcing:** Select long-lifecycle, industrial components with reliable supply chains.

Space Optimization and Form Factor Refinement

Focus on reducing physical footprint and enhancing sensor capability.

- **Integrated 3D Sensing:** Incorporate smaller 3D sensors to eliminate the **Dependence on 2D LIDAR** and detect overhanging/low-lying obstacles.
- **System Miniaturization:** Optimize component layout for maximum volume efficiency to achieve **Space Optimization**.
- **Aisle Compatibility:** Ensure the final chassis meets size requirements for common warehouse aisles, increasing **Deployment Potential**.

Enhancing Generalizability of Paths and Maps

Focus on adaptability to different environments and reducing mapping overhead.

- **Online Map Update:** Implement real-time mechanisms to update keepout zones, relaxing the rigid **Static Map Assumption**.
- **Geometry-Agnostic Poses:** Use computer vision to determine rack approach poses, reducing reliance on perfect map **Geometric Constraints**.
- **Adaptive Planning:** Develop a navigation stack that dynamically adjusts parameters, supporting **Generalizability of Paths** across different facility types.

9.3 Scalability

Scalability improvements focus on enhancing the system's ability to handle larger workloads, greater precision, and diverse environments.

Camera Resolution Upgrade

- Upgrade from the current 1920×1080 camera to a higher-resolution sensor (**4K** or **8K**) or a sensor with a higher frame rate.
- Necessary for reliable long-range barcode reading, detailed inspection, and accurate pose estimation.

Lead Screw Quality and Microstepping

- Improve the quality of the mechanical lead screw with tighter manufacturing tolerances.
- Utilize motor drivers with better **microstepping capability** for finer positional control and resolution.
- Essential for achieving sub-millimeter precision and repeatability in high-speed scanning operations.

Localization Sensor Redundancy

- Augment the current two-sensor setup with **more sensors for better localization**.
- Integrate additional sensors, such as an **IMU** and **depth cameras**, for multi-sensor data fusion.
- Provides necessary redundancy to maintain high localization accuracy when individual sensor data is noisy or unavailable, crucial for large-scale deployments.

Usage of PCB Circuitry

- Transition all integrated circuitry to custom-designed **PCB circuitry** (Printed Circuit Boards).
- Ensures **electrical consistency** and **robustness** across all manufactured units.
- Essential for reliable performance, simplified assembly, and cost-effective mass production.

Chapter 10

Innovation Highlights

10.1 Innovative Approaches

The robot uses a unique **hybrid modular-mechanical architecture** prioritizing decoupling and robustness.

- **Mechanical Decoupling:** Separation of navigation from actuation. Key is the **mechanically isolated vertical lift column**.
 - *Benefit:* Minimizes vibration, enabling **sharper images during motion** without computational compensation.
- **Decentralized Control:** Utilizes a **network of microcontroller subsystems**.
 - *Benefit:* Avoids single-point failures and provides **plug-and-play serviceability** via standardized interconnects.
- **Enhanced Stability:** A **distributed component mounting strategy** ensures balance, reducing control overhead.

10.2 Algorithms Used

The core algorithmic innovation optimizes the **Nav2** planner via map preprocessing.

Constrained Path Planning

After mapping (SLAM Toolbox), an offline **black-and-white keep-out layer** defines restricted corridors (G_{white}) and forbidden areas (S_{black}).

- **Search Space Pruning:** Feasible space is restricted to:

$$S' = S \setminus S_{\text{black}}$$

- *Benefit:* This pruning **reduces node expansions**, lowering **computational complexity** and planning time.
- **Transformation:** Global planning becomes a simpler **constrained graph traversal**, ensuring **repeatable and predictable navigation behavior**.

Chapter 11

Testing and Validation

11.1 Structure of Testing

XY Mechanism

The XY drive mechanism was evaluated for positional accuracy, repeatability, and trajectory stability in holonomic motion. The system specification requires a horizontal positioning accuracy of ± 10 cm. The robot was commanded to traverse rectangular paths with edge lengths of 1.5 m and 2 m, using mecanum wheels with a diameter of 10 cm and width of 5 cm. Across 10 repeated trials, the average positional deviation at the end of the trajectory was recorded as:

- Mean translational error: 6.8 cm
- Maximum observed error: 9 cm

In 8 out of 10 trials, the robot returned to within the ± 10 cm tolerance band of its starting position. Deviations were attributed primarily to wheel slip and minor encoder noise, but performance remained compatible with aisle-level alignment and rack localization tasks.

Z - Mechanism

The Zaxis lift mechanism uses a dual leadscrew assembly with a pitch of 2 mm per revolution, enabling high resolution vertical actuation. Under closed-loop control, the mechanism achieved:

- Vertical positioning accuracy: ± 2.0 mm
- Maximum stable velocity: 9.4 mm/s (rating: 10 mm/s)
- Repeatability across cycles: 96.7%

Guard rails mechanically stabilize the lift platform, significantly reducing lateral wobble and vibration. Drift and position loss under repeated cycles were minimal, validating suitability for continuous scanning.

QR Scanning

The QR scanning subsystem was validated using indoor printed QR tags placed at varying heights and orientations. Under nominal lighting conditions, the system achieved:

- Detection rate: 93.5%
- Successful decode accuracy: 92.1%

- Worst-case detection range: 1.25 m

Failures predominantly occurred at steep angles ($> 45^\circ$) or under excessive glare. Integration of vertical stabilization and consistent scanning velocity improved detection performance significantly.

SLAM Navigation and Obstacle Avoidance

The navigation stack utilized AMCL for pose estimation in a 2D occupancy map generated by LiDAR. With reliable localization and motion control, performance was characterized as follows:

- Mean localization error: 4.9 cm
- Maximum observed error: 13.4 cm
- Obstacle avoidance success rate: 98.2%

Guardrails and controlled wheel geometry reduced lateral oscillations during traversal, resulting in smoother trajectories and improved localization convergence.

Overall, the system operated consistently within or very close to design specifications. Horizontal positioning accuracy and QR detection rates were near-target, while vertical actuation exhibited high repeatability and minimal drift, validating the effectiveness of the mechanical stabilization strategy.

Chapter 12

User Manual Details

Safety Warnings and Precautions

Read before operating. Failure to follow instructions may result in injury or equipment damage.

General Warnings

- **DO NOT** operate without proper training and authorization.
- Keep hands, clothing, and loose accessories **AWAY** from moving parts.
- **NEVER** attempt modify electrical, mechanical, or control systems.
- Operate **ONLY** on smooth, flat surfaces as intended.
- Maintain a minimum safety perimeter of **1.5 meters** around the robot in case of unforeseen movements.

Joystick and Control Safety

- Always **VERIFY** control response before operation.
- Keep the emergency stop button(**X on Joystick**) unobstructed and accessible at all times.
- **RELEASE** joystick immediately if unexpected movement occurs.

Operational Safety

- **DO NOT** exceed maximum payload rating.
- **AVOID** steep inclines, loose gravel, wet surfaces, or steps.
- **NEVER** ride, sit or climb onto the robot enclosure or rack.

Electrical and Battery Safety

- Charge **ONLY** using approved certified chargers.
- **DO NOT** expose to water, solvents, or conductive dust.
- **DISCONNECT POWER** before servicing or changing components.

Emergency Response

- Press Emergency Stop(**X on Joystick**) immediately if robot behaves unpredictably.
- Contact a qualified technician if restart attempts fail.

Operation Guide

1. Verify battery level and ensure environment is clear.
2. Power on the robot and joystick controller.
3. Pair joystick with robot.
4. Test forward, reverse, lateral, and rotation motion slowly.
5. Use **Y on Joystick** to save map in the SLAM round.
6. Begin task operation and monitor robot movements regularly.
7. At the end of use, stop the robot, power off, and secure charging.

Troubleshooting

- Check battery, all connections, and whether Emergency Stop is engaged.
- For a delayed or unstable response, recalibrate joystick or restart system.
- If uneven translating motion is observed, inspect wheels for dirt, debris, or mechanical damage.
- During system power loss, verify connectors or charger integrity.

Chapter 13

Maintenance

Proper maintenance ensures safe, reliable operation and extends the lifespan of the robot system.

Preventive Maintenance Schedule

- Before each use, inspect wheels, verify joystick function, check battery charge, and ensure no loose structural components.
- Check for wear of sliding mechanism , tighten fasteners, test battery performance, and validate emergency stop response.
- Perform a full diagnostic calibration, replace worn tires, and inspect internal electronics with qualified personnel.

Replacement Parts

Only certified and approved components may be installed to ensure safety compliance. Some common replacement components include:

- Wheel assemblies and bearings
- 3D printed couplers
- Battery packs and charging connectors
- Joystick controller components
- Fasteners and wiring harnesses
- Camera, LiDAR

Important: Replacement of structural, electrical, or control system components must be performed by authorized technicians. Unauthorized part substitution voids warranty and may create hazards.

Storage Guidelines

- Store in a clean, dry, dust-free indoor environment.
- Keep battery charge between 40 to 70% for long-term storage (over 30 days).
- Cover the robot to prevent particulate accumulation..
- **DO NOT** stack any load on the robot chassis

Transport Guidelines

- **POWER OFF** the robot before movement/transportation.
- Use rated straps or lifting equipment when transporting or loading into vehicles.
- Avoid shock, vibration, or impacts during handling.
- Secure joystick separately to prevent accidental activation or mechanical damage.

Chapter 14

References

14.1 ROS packages

- `ldlidar_stl_ros2`, ROS2 packages for interfacing with the LiDAR, (LDLIDAR ROS2 LiDAR Driver)
- The RGB camera is interfaced using the `hk_camera` ROS 2 node (Hikvision ROS 2 Camera Driver).

14.2 Libraries Used

- `Navigation2 Stack`, `nav2_bringup` in slam and navigation launch files.
- `slam_toolbox` in slam launch file.
- `OpenCV` for the post processing of generated `.pgm` and generating keepout map.
- `CvBridge` for QR detection node.
- `pyzbar` for reading QR codes.
- `Joy` package for interfacing with Joystick, for teleop control.
- `PySerial` for communication with ESP32, in motor and lead screw control.